



TILERA[®]

***MIGRATING TO
THE TILE-GX
PROCESSOR FAMILY***

DOCUMENT RELEASE 1.10
DOC. No. UG521
SEPTEMBER 2013
TILERA CORPORATION

Copyright © 2011-2013 Tileria® Corporation. All rights reserved. Printed in the United States of America.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tileria Corporation: Tileria and the Tileria logo.

The following are trademarks of Tileria Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, TILEExpressPro-20G, TILEExpressPro-22G, iMesh, TileDirect, TILExtreme-Gx, TILExtreme-Gx Duo, TILEmpower, TILEmpower-Gx, TILEmpower-Gx72, TILEncore, TILEncorePro, TILEncore-Gx, TILEncore-Gx36, TILEncore-Gx72, TILE-Gx, TILE-Gx9, TILE-Gx16, TILE-Gx36, TILE-Gx72, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, TILE-Gx8009, TILE-Gx8016, TILE-Gx8036, TILE-Gx3036, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, iLib, TMC (Tileria Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tileria IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

This document contains advance information on Tileria products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tileria Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tileria disclaims any express or implied warranty relating to the sale and/or use of Tileria products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN "AS IS" BASIS. Tileria assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Publishing Information:

Document Number	UG521
Document Release	1.10
Date	13 September 2013

Contact Information:

Tileria Corporation
Information info@tilera.com
Web Site http://www.tileria.com

CONTENTS

PREFACE	V
 CHAPTER 1 TILE-Gx PROCESSOR FAMILY FEATURES	
1.1 Overview	1
 CHAPTER 2 SOFTWARE FEATURES FOR MDE RELEASE 4.2	
2.1 GXIO Library	3
2.2 MiCA	4
2.2.1 Crypto Acceleration	4
2.2.1.1 Protocols Supported	4
2.2.1.2 Symmetric-Key Algorithms Supported	4
2.2.1.3 Cryptographic Hash Functions Available	4
2.2.1.4 Public-Key Algorithms Available	4
2.3 mPIPE	4
2.4 TRIO	5
2.5 Changes in the Installation Process	5
2.6 Changes in tile-gcc/g++	5
2.7 Simulator in the MDE Release 4.2	5
2.8 TILE-Gx PCIe Drivers	6
2.9 Miscellaneous Changes	6
2.9.1 Tile-to-Tile DMA	6
 CHAPTER 3 COMPARISON OF 3.0 AND 4.x SOFTWARE	
3.1 Which MDE Release 3.0 Software Is Replaced?	7
3.1.1 NetIO Link Management Model Versus mPIPE Link Management Model	7
3.2 Which MDE Release 3.0 Software Is Deleted?	8
3.3 Other Significant Changes	8
3.3.1 Location Change for /proc/tile/ and /proc/sys/tile/ Directories	8
3.3.2 Mesh Networks	8
 CHAPTER 4 INITIALIZATION AND CONFIGURATION CHANGES	
4.1 Installation Process Changes	11
 CHAPTER 5 PORTING AN APPLICATION	
5.1 Run make clean and Recompile	13
5.2 Migrating from NetIO to mPIPE	13
5.2.1 Initializing the Software	13
5.2.2 Receiving Packets	13

5.2.3 Sending Packets 14

5.2.4 Extracting Packet Information 14

5.2.5 Understanding the mPIPE Packet Descriptor Structure 14

5.3 Comparing Single and Multiple Instance mPIPE Operations 18

CHAPTER 6 MDE SOFTWARE WITH NO MIGRATION ISSUES

6.1 Tools That are Identical 27

6.2 Tools That are the Same From the User Perspective 27

INDEX 29

PREFACE

About This Guide

This guide explains how to migrate your applications from the Multicore Development Environment™ (MDE) Release 3.0 to MDE Release 4.2.

Intended Audience

This guide is intended for use by system and application developers working at sites that have already been using the MDE on the TILEPro processor family and plan to use MDE on the TILE-Gx™ processor family.

If you plan to migrate from the TILE64™ to the TILE-Gx processor family, you must first migrate to the TILEPro processor family. Refer to *Migrating to the TILEPro Processor Family* (UG221).

Document Organization

This manual is organized as follows:

- [Chapter 1: TILE-Gx Processor Family Features](#)
- [Chapter 2: Software Features for MDE Release 4.2](#)
- [Chapter 4: Initialization and Configuration Changes](#)
- [Chapter 5: Porting an Application](#)
- [Chapter 6: MDE Software With No Migration Issues](#)

This guide also contains an [Index](#).

MDE Documentation

Note: `TILER_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tiler tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own [Tilera IDE Online Documentation](#), which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command [Help → Help Contents](#).

- When the online help browser is displayed, select the document [Tilera IDE Online Documentation](#) in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tilera Web site at <http://www.tilera.com>.
- E-mail questions to support@tilera.com.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
{ }	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.

Example	Description
[]	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
. . .	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.
\$	This is the system prompt for host-side commands.
#	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as <code>root</code> .
<code>command</code>	A command name appears in the Courier New font
<code>computer output</code>	Computer output appears in the Courier New font.
<i>filename</i>	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the <i>Arial</i> font.
<i>new term</i>	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
TILERA_ROOT	TILERA_ROOT is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tiler tools and examples. We recommend that you use the <code>tile-env</code> script described in <i>Getting Started with the Multicore Development Environment</i> (UG504) to set it correctly.

CHAPTER 1 TILE-Gx PROCESSOR FAMILY FEATURES

This chapter describes the major features in the TILE-Gx™ processor family that have an impact on software.

1.1 Overview

The TILE-Gx processors provides integrated high-performance interfaces that can be configured as four “double-XAUI” at 20Gbps per port (for TILE-Gx8036 only). The following interfaces are supported:

- Up to four 10Gbps Ethernet XAUI interfaces
- Up to sixteen 10/100/1000 SGMII Ethernet interfaces
- Up to three Gen2 PCIe interfaces (root or endpoint)
- Up to three 20Gbps StreamIO™ interfaces for chip-to-chip or FPGA interconnect

The TILE-Gx have much faster PCIe interfaces—as much as four times the speed from 8-lane Gen 2 PCIe. The PCIe interfaces also manage the low-level transactions in hardware in the TILE-Gx, instead of in software as on *TILEPro*. The TILE-Gx™ processor’s PCIe Controller (TRIO) provides services to integrate the Tile processors with a PCI system. Both endpoint and root complex modes are supported. Several data movement and communication models are supported simultaneously.

Applications that require a wire-speed performance, programmable operations and zero overhead on tiles make use of multicore Programmable Intelligent Packet Engine (mPIPE™). mPIPE offers packet parsing and classification, load balancing, and a packet buffer manager. Tiles communicate with I/O devices via loads and stores to MMIO pages. Data and descriptors move via cache-coherent reads and writes. A series of interrupts are delivered to tile(s) via an Interprocessor Interrupt (IPI) mechanism.

The TILE-Gx processors feature multiple new cryptographic and compression acceleration MiCA™ (Multicore iMesh Coprocessing Accelerator) engines for high speed crypto and compression operations. MiCA is an architecture that provides hardware accelerator services to applications running on Tiles.

MiCA engines provide for both symmetric and asymmetric (public key) acceleration deliver high-speed encryption/decryption and handshake processing for IPsec, SSL/TLS/DTLS, MACsec, SRTP and wireless security. The MiCA connects to TILE-Gx’s memory networks and processes requests via its memory mapped I/O interface (MMIO).

In addition to the Tile Multicore Components (TMCT™) APIs, the expanded API libraries include support for low-level I/O device control, user-level crypto acceleration, and PCIe and StreamIO data transfer APIs. Refer to the *Application Libraries Reference Manual* (UG427) for more information.

The TILE-Gx processor is supported by the following programming environments:

- Full SMP, 64-way Linux (3.4.61)

- Zero overhead Linux (ZOL) tiles
- Bare metal environment (BME)
- Erlang port

CHAPTER 2 SOFTWARE FEATURES FOR MDE

RELEASE 4.2

This chapter describes the software changes in the MDE Release 4.2:

- [Section 2.1 “GXIO Library”](#)
- [Section 2.2 “MiCA”](#)
- [Section 2.3 “mPIPE”](#)
- [Section 2.4 “TRIO”](#)
- [Section 2.5 “Changes in the Installation Process”](#)
- [Section 2.6 “Changes in tile-gcc/g++”](#)
- [Section 2.7 “Simulator in the MDE Release 4.2”](#)
- [Section 2.8 “TILE-Gx PCIe Drivers”](#)
- [Section 2.9 “Miscellaneous Changes”](#)

Some of these features require no modifications to your source code, some require minimal modifications, and some require thinking about how to change your code to achieve the best results.

2.1 GXIO Library

The GXIO library provides application control of the TILE-Gx I/O shims.

The library is designed to provide resource allocation and command queueing with the minimum amount of overhead required for interprocess protection.

The GXIO library strongly reflects the hardware MMIO control registers. Fast-path operations like sending and receiving packets are implemented entirely in userspace.

All shims with a GXIO interface have a “context” data structure (also known as a context object). This context object allows the application to use the shim. The context objects vary in function, depending on the API(s) that use them.

The GXIO library has these associated device APIs:

- mPIPE device API (`gxio/mpipe.h`)
- TRIO device API (`gxio/trio.h`)
- MiCA™ device API (`gxio/mica.h`)
- GPIO device API (`gxio/gpio.h`)

The GXIO library is documented in the *Application Libraries Reference Manual* (UG527).

2.2 MiCA

2.2.1 Crypto Acceleration

2.2.1.1 Protocols Supported

Hardware support for these protocols is available:

- IPsec (Internet Protocol Security)
- SSL/TLS/DTLS (Secure Sockets Layer/Transport Layer Security/Datagram Transport Layer Security)
- SRTP (Secure Real-time Transport Protocol)
- MACsec (802.1AE, the IEEE MAC Security standard)
- 3GPP (3rd Generation Partnership Project)

2.2.1.2 Symmetric-Key Algorithms Supported

- Advanced Encryption Standard (AES) with 128-, 192-, and 256-bit keys
- DES (Data Encryption Standard)
- Triple DES (3DES) for block ciphers
- KASUMI (block cipher used in the 3GPP mobile communications network)
- Snow (word-based synchronous stream cipher)

2.2.1.3 Cryptographic Hash Functions Available

- SHA-1 (Secure Hash Algorithm)
- SHA-2 (SHA-224, SHA-256, SHA-384, SHA-512)
- MD5 (Message Digest Algorithm)
- HMAC (Hash-based Message Authentication Code), available in the IPsec tokens

2.2.1.4 Public-Key Algorithms Available

- RSA (Rivest, Shamir and Adleman)
- DSA (Digital Signature Algorithm)
- Diffie-Hellman key exchange
- Elliptic curve cryptography (ECC)
- Onboard true random number generator (RNG)

2.3 mPIPE

mPIPE™ is Tiler's Multicore Programmable Intelligent Packet Engine for the TILE-Gx family of processors. mPIPE provides line rate services for the packet interfaces.

mPIPE has no IPP and requires no .hvc file entries. The hypervisor configures the driver automatically, and any subsequent configuration is done by means of C APIs documented in the *Application Libraries Reference Manual* (UG527).

At version 4.2 the MDE supports the TILE-Gx72™ processor which provides two mPIPE instances. To learn about the differences between using mPIPE in single instance versus multiple instance modes, see XXX.

2.4 TRIO

TRIO (transactional I/O) handles the direct memory access (DMA) data movement and buffer management between the Tile memory system and the transaction-oriented (that is, PCIe and StreamIO) subsystems.

For more information, see the *MDE TRIO Programmer's Guide* (UG503).

2.5 Changes in the Installation Process

The installation process for MDE Release 4.2 has been changed to make it more user-friendly.

The primary changes are:

- The most important RPMs (approximately 160 of them) are now in a separate, “basic” Tile tarball that can be easily extracted.
- Approximately 4,000 additional RPMs are available in a separate tarball. You can use Yum, an automatic package installer for RPMs, to install the RPMs you choose.
- If you want all of the RPMs, you can extract what is called the “full” Tile tarball.
- There is no longer a Source tarball. Some of the source files are now in the MDE tarball. Others, such as Linux and gcc, are contained in .get files. You can run the .get files to extract the sources.
- Sites can perform either an `initramfs` boot or an `upstart` boot.

2.6 Changes in tile-gcc/g++

Most of the intrinsics have changed.

In addition, some TILE-Gx instructions clamp their results. For these instructions, the compiler supports two variants of instructions.

For more information, see the section on “Using Intrinsics” in the chapter “Using the Optimization Features of the tile-gcc/g++ Compiler” of the *Multicore Development Environment Optimization Guide* (UG515).

2.7 Simulator in the MDE Release 4.2

The MDE Release 4.0 simulator has a detailed model of the processor pipeline, cache subsystem, network, memory controller, and DDR3 timing.

The core and memory run in separate clock domains, and DDR3 timing is modeled at the device command level.

There are some things that the simulator does not model. Most are either features that software does not currently exercise, or second-order microarchitectural details.

A few are notable:

- The simulator does not model the instruction prefetcher (instead, it is pessimistic), and does not model way prediction, and inexactly models branch prediction (instead, it is optimistic).
- The simulator does not model reduced sharer vectors and inval compression; it is accurate for TILE-Gx36™, but optimistic for TILE-Gx64™ and TILE-Gx100™ configurations.

2.8 TILE-Gx PCIe Drivers

All inputs and outputs performed on the TILE-Gx processor family are direct-to-cache. There is no provision for the to-memory mode (software-managed coherency) that was available for the *TILEPro*[™] processor family.

Two drivers enable linking from the TILE-Gx to devices. The endpoint driver configures the TILE-Gx link as the “device”, while a root complex driver enables the TILE-Gx link to be the “master”.

A host subset driver sits on top of the root complex driver and communicates with the TILE-Gx endpoint on the other side to implement chip-to-chip communication. This process can be thought of as equivalent to a NIC or SATA controller driver, with the TILE-Gx endpoint on the other side as the device being controlled.

The chip-to-chip API sits on top of the endpoint and host subset drivers.

Various PCIe features and the drivers and APIs exercise the following:

- Memory mapped I/O interface (MMIO) and Programmed input/output (PIO)
- Push and pull DMA
- Scatter queues
- Interrupts

2.9 Miscellaneous Changes

2.9.1 Tile-to-Tile DMA

The TILE-Gx processor family does not support tile-to-tile DMA.

CHAPTER 3 COMPARISON OF 3.0 AND 4.X SOFTWARE

This chapter compares the components of the MDE 3.0 release and the MDE 4.0 release:

- [Section 3.1 “Which MDE Release 3.0 Software Is Replaced?”](#)
- [Section 3.2 “Which MDE Release 3.0 Software Is Deleted?”](#)
- [Section 3.3 “Other Significant Changes”](#)

See also [Chapter 6: MDE Software With No Migration Issues](#).

3.1 Which MDE Release 3.0 Software Is Replaced?

[Table 3-1](#) shows which MDE Release 3.0 software components are replaced by MDE Release 4.0 software components.

Table 3-1. MDE 3.0 Software That Is Replaced

MDE 3.0 Component	Replaced By	Where Documented
IPP/EPP drivers	mPIPE driver	<i>MDE mPIPE Programmer's Guide</i> (UG506)
netio	mPIPE	<i>MDE mPIPE Programmer's Guide</i> (UG506)
netio-stat	mpipe-stat	<i>MDE mPIPE Programmer's Guide</i> (UG506)
PCIe driver	TRIO driver	<i>MDE TRIO Programmer's Guide</i> (UG503)
XGbE/GbE driver	mPIPE driver	<i>MDE mPIPE Programmer's Guide</i> (UG506)
/dev/hugetlb	/dev/hugepages	<i>Programming the Tile Processor</i> (UG505)

3.1.1 NetIO Link Management Model Versus mPIPE Link Management Model

There are significant similarities between the NetIO link management model and the mPIPE link management model. However, the features provided by the mPIPE hardware and the mPIPE GXIO library are significantly different from those provided by NetIO. Thus, the mPIPE link management operations are not exactly equivalent to the NetIO operations, and you should carefully read the mPIPE documentation.

For example, the mPIPE link management model uses a permission scheme for increased security. There is no counterpart in NetIO. Another difference is: By default, a link is taken down on link close and application exit.

3.2 Which MDE Release 3.0 Software Is Deleted?

Table 3-2 shows which MDE Rel.3.0 software components are deleted and are not replaced.

Table 3-2. MDE 3.0 Software That Is Deleted With No 4.0 Counterpart

MDE 3.0 Component	Description
HPI driver	Not supported in TILE-Gx processors.

3.3 Other Significant Changes

3.3.1 Location Change for /proc/tile/ and /proc/sys/tile/ Directories

To conform with Linux community standards, some of the files in the /proc/tile/ and /proc/sys/tile/ directories have changed locations:

- /proc/tile/board is now several one-line files in the /sys/hypervisor/board directory.
- /proc/tile/dataplane is now /sys/devices/system/cpu/dataplane.
- /proc/tile/grid is now two files: /sys/devices/system/cpu/chip_width and /sys/devices/system/cpu/chip_height.
- /proc/tile/hv is now two files: /sys/hypervisor/version and /sys/hypervisor/config_version.
- /proc/tile/hvconfig is now /sys/hypervisor/hvconfig.
- /proc/sys/tile/crashinfo is now /proc/sys/debug/exception-trace.

3.3.2 Mesh Networks

The Static Network (STN), an on-chip communication device for the TILEPro processor family, is not available in the TILE-Gx processor family.

The iMesh for the TILEPro processor family provided five dynamic networks plus the STN. These include: the User Dynamic Network (UDN); I/O Dynamic Network (IDN); Memory Dynamic Network (MDN); Coherence Dynamic Network (CDN); and Tile Dynamic Network (TDN).

In contrast, the iMesh for the TILE-Gx processor family has five dynamic networks—only one (the UDN) is the same as for TILEPro:

- UDN (User Dynamic Network)
- IDN (Internal Dynamic Network)
- QDN (reQuest Dynamic Network)
- RDN (Response Dynamic Network)
- SDN (Share Dynamic Network)

Some performance counters refer to these networks.

For the TILEPro processor family, the IDN was called the I/O Dynamic Network. It was not available to applications.

For the TILE-Gx processor family, the IDN is called the Internal Dynamic Network, and it is expected to be available to applications.

CHAPTER 4 INITIALIZATION AND CONFIGURATION CHANGES

This chapter describes changes in initialization and configuration for sites migrating to the TILE-Gx™ processor family.

4.1 Installation Process Changes

There is no longer a `src` tarball. Some of the source files are now in the MDE tarball. Others, such as Linux and `gcc`, are contained in `.get` files. Run the `.get` files to extract the sources.

The applications tarball is no longer shipped as part of the MDE distribution.

For information on installing the MDE, see *Getting Started with the Multicore Development Environment* (UG504).

CHAPTER 5 PORTING AN APPLICATION

This chapter provides information about changes that are required for your code to run. The changes described are the minimal necessary changes you must make:

5.1 Run make clean and Recompile

You need to run `make clean` (to clear out old executable files) and then recompile whenever the MDE software is upgraded.

In addition, if you have applications written for MDE 2.0 that run on the TILE64 processor, you will need to run `make clean` and then recompile the applications to run on the TILEPro processor family. If you write applications for MDE 2.0 that run on the TILEPro processor family, you must run `make clean` and recompile to run on the TILE64 processor.

If you mix stale `.o` files from different processor families, you will receive a message such as: "Could not read symbols: File in wrong format."

5.2 Migrating from NetIO to mPIPE

The following section explain aspects of migrating applications from using NetIO to the new mPIPE architecture.

5.2.1 Initializing the Software

Use the mPIPE hardware through a memory-mapped I/O interface. You initialize mPIPE by accessing the virtual memory (VA) area that is associated with an I/O TLB. You can access this VA from all Tiles. Unlike NetIO, you only have to initialize mPIPE initialization once. Worker Tiles only need to perform packet receive/transmit operations using the gxio API.

To learn the proper method for initializing mPIPE blocks, refer to the example file `$TILERA_ROOT/examples/mpipe/forward/app.c`.

5.2.2 Receiving Packets

The mPIPE routine which corresponds to the `netio_get_packet()` routine is the `gxio_mpipe_iqueue_get()` routine. This routine gets one packet each time you call the API. Use the `gxio_mpipe_iqueue_peek()` and `gxio_mpipe_iqueue_try_peek()` routines to return packet descriptors in the current ingress queue. If your application needs more performance, use these routines in a process to handle multiple packets at one time. For example, add a loop in the packet processing routine and give a flag to indicate if that a particular packet can proceed to next block. Use the bit flag "nothing to be sent (ns)" in the eDMA descriptor; your application only needs to raise this flag so that packets are dropped by the egress DMA hardware.

5.2.3 Sending Packets

The mPIPE routine which corresponds to the `netio_send_packet()` routine is `gxio_mpipe_equeue_put()`. A call to the `gxio_mpipe_equeue_put()` routine puts a single packet into the eDMA ring. You can also process multiple packets at a time, using the `gxio_mpipe_equeue_reserve()` and `gxio_mpipe_equeue_put_at()` routines.

5.2.4 Extracting Packet Information

NETIO has APIs to extract packet information from metadata. For example, if you want to pull a memory address where packet data is stored, call the `NETIO_PKT_L2_DATA()` query function. For mPIPE, it is one of field parameters in the packet descriptor structure.

With NetIO, you can get a packet using this code:

```
netio_queue_t *queue;
netio_pkt_t *packet;
netio_get_packet(queue, packet);
char *L2_data= NETIO_PKT_L2_DATA(...);
```

To perform the same operation for mPIPE, use:

```
gxio_mpipe_idesc_t* idesc;
gxio_mpipe_iqueue_try_peek(iqueue, &idesc)
char *L2_data=idesc->va
```

5.2.5 Understanding the mPIPE Packet Descriptor Structure

The structure `MPIPE_PDESC_t` defines the mPIPE packet descriptor, located in this file:

`$TILERA_ROOT/tile/usr/include/arch/mpipe_shm.h`

The correlation between NetIO functions which provide pointer to packet ingress data and mPIPE pointers to data.

Table 5-1. Comparing NetIO and mPIPE Packet Data Pointer Routines

NetIO	mPIPE
NETIO_PKT_L2_DATA	idesc->va
NETIO_PKT_L2_LENGTH	idesc->l2_size
NETIO_PKT_VLAN_ID	idesc->custom1, Bit 0-15
NETIO_PKT_FLOW_HASH	idesc->custom0
NETIO_PKT_ETHERTYPE	idesc->custom1, Bit 16-31

You should extract most of packet information, since default mPIPE classifier has cycling limitations to achieve a high packet throughput performance.

If your application requires a customized packet header, use the mPIPE classifier to create a custom packet descriptor by consuming CPU cycles of the classifier engine.

You can also examine a packet after the packet arrives at a Tile, in a manner similar to what NETIO does in the IPP hypervisor, since this is all just software processing.

Table 5-2 displays the mPIPE packet descriptor structure details.

Table 5-2. mPIPE Packet Descriptor Details

Word Number	Designation	Size (bits)	Description
Word 0	notif_ring	8	Notification ring into which this packet descriptor is written. Typically, this is written by the load balancer, but this value can be overridden by the classification program if NR is asserted.
Word 0	channel	5	Source channel for this packet. Written by mPIPE DMA hardware.
Word 0	__reserved_0	1	Reserved.
Word 0	me	1	MAC Error, generated by the MAC interface. Asserted if there was an overrun of the MAC's receive FIFO. This condition generally only occurs if the mPIPE clock is running too slowly.
Word 0	tr	1	Truncation Error, written by the iDMA hardware. Asserted if packet was truncated due to insufficient space in iPkt buffer.
Word 0	l2_size	14	Written by the iDMA hardware. Indicates the number of bytes written to Tile memory. In general, this is the actual size of the packet as received from the MAC. But if the packet is truncated due to running out of buffers or due to the iPkt buffer filling up, then the L2_SIZE will be reduced to reflect the actual number of valid bytes written to Tile memory.
Word 0	ce	1	CRC Error, generated by the MAC. Asserted if MAC indicated an L2 CRC error or other L2 error (bad length etc.) on the packet.
Word 0	ct	1	Cut Through, written by the iDMA hardware. Asserted if packet was not completely received before being sent to classifier. L2_Size indicates number of bytes received so far.
Word 0	bucket_id	13	Bucket identifier, written by the classification program. Used by the load balancer to select the ring into which this packet descriptor is written.
Word 0	__reserved_1	3	Reserved.
Word 0	cs	1	Checksum, written by classification program. When 1, the checksum engine performs checksum operations based on the CSUM_SEED, CSUM_START, and CSUM_BYTES fields. The result appears in CSUM_VAL.
Word 0	nr	1	Notification Ring Select, written by the classification program. When 1, the NotifRingIDX is set by classification program rather than being set by load balancer.
Word 0	dest	2	Destination, written by classification program. Indicates whether packet and descriptor should both be dropped, both be delivered, or only the descriptor should be delivered.
Word 0	sq	1	General Purpose Sequence Number Enable, written by the classification program. When set to 1, the GP_SQN_SEL field contains the sequence number selector and the GP_SQN field contains the associated sequence number. When clear, the GP_SQN field is left intact and be used as "Custom" bytes.

Table 5-2. mPIPE Packet Descriptor Details (continued)

Word Number	Designation	Size (bits)	Description
Word 0	ts	1	TimeStamp Enable, which enables TimeStamp insertion. When clear, time-stamp field may be filled with custom data by classifier. When set, hardware inserts the timestamp when the start of packet is received from the MAC.
Word 0	ps	1	Packet Sequence Number Enable, which enables PacketSQN insertion. When clear, PacketSQN field may be filled with custom data by classifier. When set, hardware inserts the packet sequence number when the packet descriptor is written to a notification ring.
Word 0	be	1	Buffer Error, written by the iDMA hardware. Asserted if iDMA ran out of buffers while writing the packet. Software must still return any buffer descriptors whose C field indicates a valid descriptor was consumed.
Word 0	ctr0	5	Counter, written by the classification program. The associated counter is incremented when the packet is sent.
Word 0	__reserved_2	3	Reserved.
Word 1	ctr1	5	Counter2, written by the classification program. The associated counter is incremented when the packet is sent.
Word 1	__reserved_3	3	Reserved.
Word 1	csum_start	8	Checksum starting byte, written by classification program. This indicates the start byte for checksum, relative to the first byte from the MAC.
Word 1	csum_seed_val	16	Checksum seed written by classification program. Overwritten with resultant checksum if CS bit is asserted. The endianness of the CSUM value bits when viewed by Tile software match the packet byte order. That is, bits[7:0] of the resulting checksum value correspond to earlier (more significant) bytes in the packet. To avoid classifier software from having to byte swap the CSUM_SEED, the iDMA checksum engine byte swaps the classifier's result before seeding the checksum calculation. Thus, the CSUM_START byte of packet data is added to bits[15:8] of the CSUM_SEED field generated by the classifier. This byte swap will be visible to Tile software if the CS bit is clear.
Word 1	custom0	32	A custom value, written by the classification program. Not interpreted by mPIPE hardware.
Word 2	custom1	64	A custom value, written by the classification program. Not interpreted by mPIPE hardware.
Word 3	custom2	64	A custom value, written by the classification program. Not interpreted by mPIPE hardware.
Word 4	custom3	64	A custom value, written by the classification program. Not interpreted by mPIPE hardware.
Word 5	gp_sqn	16	Sequence number applied when packet is distributed. The classifier selects which sequence number is to be applied by writing the 13-bit SQN-selector into this field.

Table 5-2. mPIPE Packet Descriptor Details (continued)

Word Number	Designation	Size (bits)	Description
Word 5	packet_sqn	48	Packet sequence number, written by notification hardware. The packet sequence number is incremented for each packet that wasn't dropped.
Word 6	time_stamp_ns	32	Timestamp nanosecond value, written by hardware when the start-of-packet is received by the mPIPE from the MAC. This is the nanoseconds part of the packet timestamp.
Word 6	time_stamp_sec	32	Timestamp second value, written by hardware when the start-of-packet is received by the mPIPE from the MAC. This is the seconds part of the packet timestamp.
Word 6	va	42	Virtual address. Must be sign extended by consumer.
Word 6	__reserved_4	6	Reserved.
Word 7	stack_idx	5	Index of the buffer stack to which this buffer belongs.
Word 7	__reserved_5	3	Reserved.
Word 7	inst	1	Instance ID. For devices that support more than one mPIPE instance, this field indicates the buffer owner. If the INST field does not match the mPIPE's instance number when a packet is egressed, buffers with HWB set will be returned to the other mPIPE instance.
Word 7	__reserved_6	1	Reserved.
Word 7	hwb	1	Hardware buffer, always set to one by hardware in iDMA packet descriptors. For eDMA, indicates whether the buffer will be released to the buffer stack manager. When 0, software is responsible for releasing the buffer.
Word 7	size	3	Encoded size of buffer. Set by the ingress hardware for iDMA packet descriptors. For eDMA descriptors, indicates the buffer size if .c indicates a chained packet. If an eDMA descriptor is not chained and the .hwb bit is not set, this field is ignored and the size is specified by the xfer_size field. Values are: 0 = 128 bytes 1 = 256 bytes 2 = 512 bytes 3 = 1024 bytes 4 = 1664 bytes 5 = 4096 bytes 6 = 10368 bytes 7 = 16384 bytes
Word 7	c	2	Chaining configuration for the buffer. Indicates that an ingress packet or egress command is chained across multiple buffers, with each buffer's size indicated by the .size field.

5.3 Comparing Single and Multiple Instance mPIPE Operations

MDE version 4.2 the supports the TILE-Gx72™ processor which provides two mPIPE instances. To convey the differences between forwarding packets using a single instance or multiple mPIPE instances, there are example applications available at the location:

\$TILERA_ROOT/examples/mpipe/forward/

Additional information on these examples is available in the following locations:

- “Chapter 9: Programming Multiple mPIPE Instances” in the *MDE mPIPE Programmer’s Guide* (UG506)
- Section 3.4 “Single-Instance Forward Example” in the *MDE mPIPE Programmer’s Guide* (UG506)
- Section 3.5 “Multiple mPIPE Instance Forward Example” in the *MDE mPIPE Programmer’s Guide* (UG506)
- “mpipe/forward/app.c” in the “Example Documentation” chapter of the *Application Libraries Reference Manual* (UG527)

Table 5-3 compares and contrasts the operations in the single-instance mPIPE forwarding example application (`app.c`) and the multiple-instance mPIPE example application (`app_gx8072.c`). The table includes highlighting for the major additions or changes associated with the multiple-instance methods.

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code

Single-instance mPIPE (app.c)	Multi-instance mPIPE (app_gx8072.c)
<pre>// Copyright 2013 Tiler Corporation. All Rights Reserved. // #include <assert.h> #include <stdio.h> #include <stdlib.h> #include <string.h> #include <unistd.h> #include <pthread.h> #include <sys/mman.h> #include <sys/dataplane.h> #include <tmc/alloc.h> #include <arch/atomic.h> #include <arch/sim.h> #include <gxio/mpipe.h> #include <tmc/cpus.h> #include <tmc/mem.h> #include <tmc/spin.h> #include <tmc/sync.h> #include <tmc/task.h> // Align "p" mod "align", assuming "p" is a "void*". #define ALIGN(p, align) do { (p) += -(long)(p) & ((align) - 1); } while(0) // Maximum "batch" size. #define BATCH 16 // Help synchronize thread creation. static tmc_sync_barrier_t sync_barrier; static tmc_spin_barrier_t spin_barrier;</pre>	<pre>// Copyright 2013 Tiler Corporation. All Rights Reserved. // #include <assert.h> #include <stdio.h> #include <stdlib.h> #include <string.h> #include <unistd.h> #include <pthread.h> #include <sys/mman.h> #include <sys/dataplane.h> #include <tmc/alloc.h> #include <arch/atomic.h> #include <arch/sim.h> #include <gxio/mpipe.h> #include <tmc/cpus.h> #include <tmc/mem.h> #include <tmc/spin.h> #include <tmc/sync.h> #include <tmc/task.h>8 // Align "p" mod "align", assuming "p" is a "void*". #define ALIGN(p, align) do { (p) += -(long)(p) & ((align) - 1); } while(0) // Maximum "batch" size. #define BATCH 16 // Help synchronize thread creation. static tmc_sync_barrier_t sync_barrier; static tmc_spin_barrier_t spin_barrier;</pre>

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code (continued)

Single-instance mPIPE (app.c) (continued)	Multi-instance mPIPE (app_gx8072.c) (continued)
<pre> // True if "--flip" was specified. static bool flip; // True if "--jumbo" was specified. static bool jumbo; // The number of packets to process. static int num_packets = 1000; // The flag to indicate packet forward is done. static volatile bool done = false; // The number of workers to use. #ifdef NUM_WORKERS static unsigned int num_workers = NUM_WORKERS; #else static unsigned int num_workers = 1; #endif // The number of entries in the equeue ring - 2K (2048). static unsigned int equeue_entries = GXIO_MPIPE_EQUEUE_ENTRY_2K; // The initial affinity. static cpu_set_t cpus; // The mPIPE instance. static int instance; // The mpipe context (shared by all workers). static gxio_mpipe_context_t context_body; static gxio_mpipe_context_t* const context = &context_body; // The ingress queues (one per worker). static gxio_mpipe_iqueue_t** iqueues; // The egress queue (shared by all workers). static gxio_mpipe_equeue_t equeue_body; static gxio_mpipe_equeue_t* const equeue = &equeue_body; // The total number of packets forwarded by all workers. // Reserve a cacheline for "total" to eliminate the false sharing. #define total total64.v struct { volatile unsigned long v __attribute__ ((aligned(CHIP_L2_LINE_SIZE()))); } total64 = { 0 }; // Help check for errors. #define VERIFY(VAL, WHAT) \ do { \ long long __val = (VAL); \ if (__val < 0) \ tmc_task_die("Failure in '%s': %lld: %s.", \ (WHAT), __val, gxio_strerror(__val)); \ } while (0) // Allocate memory for a buffer stack and its buffers, initialize the // stack, and push buffers onto it. // static void create_stack(gxio_mpipe_context_t* context, int stack_idx, gxio_mpipe_buffer_size_enum_t buf_size, int num_buffers) { </pre>	<pre> // True if "--flip" was specified. static bool flip; // True if "--jumbo" was specified. static bool jumbo; // The number of packets to process. static int num_packets = 1000; // The flag to indicate packet forward is done. static volatile bool done = false; // The number of workers to use. #ifdef NUM_WORKERS static unsigned int num_workers = NUM_WORKERS; #else static unsigned int num_workers = 1; #endif // The number of entries in the equeue ring - 2048. static unsigned int equeue_entries = GXIO_MPIPE_EQUEUE_ENTRY_2K; // The initial affinity. static cpu_set_t cpus; enum { // Ingress link. LINK_INGRESS = 0, // Egress link. LINK_EGRESS, // Total Link count. LINK_MAX }; // The mpipe contexts of ingress and egress links, // initialized as all NULL. static gxio_mpipe_context_t* context[LINK_MAX] = { NULL }; // The ingress queues (one per worker). static gxio_mpipe_iqueue_t** iqueues; // The egress queue (shared by all workers). static gxio_mpipe_equeue_t equeue_body; static gxio_mpipe_equeue_t* const equeue = &equeue_body; // The total number of packets forwarded by all workers. // Reserve a cacheline for "total". #define total total64.v struct { volatile unsigned long v __attribute__ ((aligned(CHIP_L2_LINE_SIZE()))); } total64 = { 0 }; // Help check for errors. #define VERIFY(VAL, WHAT) \ do { \ long long __val = (VAL); \ if (__val < 0) \ tmc_task_die("Failure in '%s': %lld: %s.", \ (WHAT), __val, gxio_strerror(__val)); \ } while (0) // Allocate memory for a buffer stack and its buffers, initialize the // stack, and push buffers onto it. // static void create_stack(gxio_mpipe_context_t* context, int stack_idx, gxio_mpipe_buffer_size_enum_t buf_size, int num_buffers) { </pre>

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code (continued)

Single-instance mPIPE (app.c) (continued)	Multi-instance mPIPE (app_gx8072.c) (continued)
<pre> int result; // Extract the actual buffer size from the enum. size_t size = gxio_mpipe_buffer_size_enum_to_buffer_size(buf_size); // Compute the total bytes needed for the stack itself. size_t stack_bytes = gxio_mpipe_calc_buffer_stack_bytes(num_buffers); // Round up so that the buffers will be properly aligned. stack_bytes += -(long)stack_bytes & (128 - 1); // Compute the total bytes needed for the stack plus the buffers. size_t needed = stack_bytes + num_buffers * size; // Allocate up to 16 pages of the smallest suitable pagesize. tmc_alloc_t alloc = TMC_ALLOC_INIT; tmc_alloc_set_pagesize(&alloc, needed / 16); size_t pagesize = tmc_alloc_get_pagesize(&alloc); int pages = (needed + pagesize - 1) / pagesize; void* mem = tmc_alloc_map(&alloc, pages * pagesize); if (mem == NULL) tmc_task_die("Could not allocate buffer pages."); // Initialize the buffer stack. result = gxio_mpipe_init_buffer_stack(context, stack_idx, buf_size, mem, stack_bytes, 0); VERIFY(result, "gxio_mpipe_init_buffer_stack()"); // Register the buffer pages. for (int i = 0; i < pages; i++) { result = gxio_mpipe_register_page(context, stack_idx, mem + i * pagesize, pagesize, 0); VERIFY(result, "gxio_mpipe_register_page()"); } // Push the actual buffers. mem += stack_bytes; for (int i = 0; i < num_buffers; i++) { gxio_mpipe_push_buffer(context, stack_idx, mem); mem += size; } // The main function for each worker thread. // static void* main_aux(void* arg) { int result; int rank = (long)arg; // Bind to a single cpu. result = tmc_cpus_set_my_cpu(tmc_cpus_find_nth_cpu(&cpus, rank)); VERIFY(result, "tmc_cpus_set_my_cpu()"); mlockall(MCL_CURRENT); tmc_sync_barrier_wait(&sync_barrier); tmc_spin_barrier_wait(&spin_barrier); result = set_dataplane(DP_DEBUG); VERIFY(result, "set_dataplane()"); tmc_spin_barrier_wait(&spin_barrier); if (rank == 0) </pre>	<pre> int result; // Extract the actual buffer size from the enum. size_t size = gxio_mpipe_buffer_size_enum_to_buffer_size(buf_size); // Compute the total bytes needed for the stack itself. size_t stack_bytes = gxio_mpipe_calc_buffer_stack_bytes(num_buffers); // Round up so that the buffers will be properly aligned. stack_bytes += -(long)stack_bytes & (128 - 1); // Compute the total bytes needed for the stack plus the buffers. size_t needed = stack_bytes + num_buffers * size; // Allocate up to 16 pages of the smallest suitable pagesize. tmc_alloc_t alloc = TMC_ALLOC_INIT; tmc_alloc_set_pagesize(&alloc, needed / 16); size_t pagesize = tmc_alloc_get_pagesize(&alloc); int pages = (needed + pagesize - 1) / pagesize; void* mem = tmc_alloc_map(&alloc, pages * pagesize); if (mem == NULL) tmc_task_die("Could not allocate buffer pages."); // Initialize the buffer stack. result = gxio_mpipe_init_buffer_stack(context, stack_idx, buf_size, mem, stack_bytes, 0); VERIFY(result, "gxio_mpipe_init_buffer_stack()"); // Register the buffer pages. for (int i = 0; i < pages; i++) { result = gxio_mpipe_register_page(context, stack_idx, mem + i * pagesize, pagesize, 0); VERIFY(result, "gxio_mpipe_register_page()"); } // Push the actual buffers. mem += stack_bytes; for (int i = 0; i < num_buffers; i++) { gxio_mpipe_push_buffer(context, stack_idx, mem); mem += size; } // The main function for each worker thread. // static void* main_aux(void* arg) { int result; int rank = (long)arg; // Bind to a single cpu. result = tmc_cpus_set_my_cpu(tmc_cpus_find_nth_cpu(&cpus, rank)); VERIFY(result, "tmc_cpus_set_my_cpu()"); mlockall(MCL_CURRENT); tmc_sync_barrier_wait(&sync_barrier); tmc_spin_barrier_wait(&spin_barrier); result = set_dataplane(DP_DEBUG); VERIFY(result, "set_dataplane()"); tmc_spin_barrier_wait(&spin_barrier); if (rank == 0) </pre>

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code (continued)

Single-instance mPIPE (app.c) (continued)	Multi-instance mPIPE (app_gx8072.c) (continued)
<pre> if (rank == 0) { // HACK: Pause briefly, to let everyone finish passing the barrier. for (int i = 0; i < 10000; i++) __insn_mfspr(SPR_PASS); // Allow packets to flow. sim_enable_mpipe_links(instance, -1); } // Forward packets. gxio_mpipe_iqueue_t* iqueue = iqueues[rank]; // Local version of "total". The optimization is to save a global variable // load (most likely in L3) per iteration and share invalidation. unsigned long local_total = 0; while (local_total < num_packets num_packets < 0) { // Wait for packets to arrive. gxio_mpipe_idesc_t* idescs; int n = gxio_mpipe_iqueue_try_peek(iqueue, &idescs); if (n > BATCH) n = BATCH; else if (n <= 0) { if (done) goto L_done; continue; } // Prefetch the actual idescs. tmc_mem_prefetch(idescs, n * sizeof(*idescs)); gxio_mpipe_edesc_t edescs[BATCH]; for (int i = 0; i < n; i++) { gxio_mpipe_idesc_t* idesc = &idescs[i]; gxio_mpipe_edesc_t* edesc = &edescs[i]; // Prepare to forward (or drop). gxio_mpipe_edesc_copy_idesc(edesc, idesc); // Drop "error" packets (but ignore "checksum" problems). if (idesc->be idesc->me idesc->tr idesc->ce) edesc->ns = 1; } if (flip) { for (int i = 0; i < n; i++) { if (!edescs[i].ns) { void* start = gxio_mpipe_idesc_get_l2_start(&idescs[i]); size_t length = gxio_mpipe_idesc_get_l2_length(&idescs[i]); // Prefetch entire packet (12 bytes would be sufficient). tmc_mem_prefetch(start, length); } } } } </pre>	<pre> if (rank == 0) { // HACK: Pause briefly, to let everyone finish passing the barrier. for (int i = 0; i < 10000; i++) __insn_mfspr(SPR_PASS); // Allow packets to flow. sim_enable_mpipe_links(0, -1); sim_enable_mpipe_links(1, -1); } // Forward packets. gxio_mpipe_iqueue_t* iqueue = iqueues[rank]; // Local version of "total". The optimization is to save a global variable // load (most likely in L3) per iteration and share invalidation. unsigned long local_total = 0; while (local_total < num_packets num_packets < 0) { // Wait for packets to arrive. gxio_mpipe_idesc_t* idescs; int n = gxio_mpipe_iqueue_try_peek(iqueue, &idescs); if (n > BATCH) { n = BATCH; } else if (n <= 0) { if (done) goto L_done; continue; } // Prefetch the actual idescs. tmc_mem_prefetch(idescs, n * sizeof(*idescs)); gxio_mpipe_edesc_t edescs[BATCH]; for (int i = 0; i < n; i++) { gxio_mpipe_idesc_t* idesc = &idescs[i]; gxio_mpipe_edesc_t* edesc = &edescs[i]; // Prepare to forward (or drop). gxio_mpipe_edesc_copy_idesc(edesc, idesc); // Drop "error" packets (but ignore "checksum" problems). if (idesc->be idesc->me idesc->tr idesc->ce) edesc->ns = 1; } if (flip) { for (int i = 0; i < n; i++) { if (!edescs[i].ns) { void* start = gxio_mpipe_idesc_get_l2_start(&idescs[i]); size_t length = gxio_mpipe_idesc_get_l2_length(&idescs[i]); // Prefetch entire packet (12 bytes would be sufficient). tmc_mem_prefetch(start, length); } } } } </pre>

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code (continued)

Single-instance mPIPE (app.c) (continued)	Multi-instance mPIPE (app_gx8072.c) (continued)
<pre> for (int i = 0; i < n; i++) { if (!edescs[i].ns) { void* start = gxio_mpipe_idesc_get_l2_start(&idescs[i]); // Flip source/dest mac addresses. unsigned char tmac[6]; void* dmac = start + 0; void* smac = start + 6; memcpy(tmac, dmac, 6); memcpy(dmac, smac, 6); memcpy(smac, tmac, 6); } } // Flush. __insn_mf(); } // Reserve slots. NOTE: This might spin. long slot = gxio_mpipe_equeue_reserve_fast(equeue, n); // Egress the packets. for (int i = 0; i < n; i++) gxio_mpipe_equeue_put_at(equeue, edescs[i], slot + i); // Consume the packets. for (int i = 0; i < n; i++) gxio_mpipe_iqueue_consume(iqueue, &idescs[i]); // Count packets (atomically). Let the local_total be the new total // value after the atomic add. Note: there is no explicit load to "total". local_total = arch_atomic_add(&total, n) + n; } // Forwarding is done! done = true; // Make done visible to all workers. __insn_mf(); L_done: return (void*)NULL; } int main(int argc, char** argv) { int result; char* link_name = "gbe0"; // Parse args. for (int i = 1; i < argc; i++) { char* arg = argv[i]; // --link <link_name>, link_name is for both ingress and egress. if (!strcmp(arg, "--link") && i + 1 < argc) { link_name = argv[++i]; } } </pre>	<pre> for (int i = 0; i < n; i++) { if (!edescs[i].ns) { void* start = gxio_mpipe_idesc_get_l2_start(&idescs[i]); // Flip source/dest mac addresses. unsigned char tmac[6]; void* dmac = start + 0; void* smac = start + 6; memcpy(tmac, dmac, 6); memcpy(dmac, smac, 6); memcpy(smac, tmac, 6); } } // Flush. __insn_mf(); } // Reserve slots. NOTE: This might spin. long slot = gxio_mpipe_equeue_reserve_fast(equeue, n); // Egress the packets. for (int i = 0; i < n; i++) gxio_mpipe_equeue_put_at(equeue, edescs[i], slot + i); // Consume the packets. for (int i = 0; i < n; i++) gxio_mpipe_iqueue_consume(iqueue, &idescs[i]); // Count packets (atomically). Let the local_total be the new total // value after the atomic add. Note: there is no explicit load to "total". local_total = arch_atomic_add(&total, n) + n; } // Forwarding is done! done = true; // Make done visible to all workers. __insn_mf(); L_done: return (void*)NULL; } int main(int argc, char** argv) { int result; int instance; char* link_name_in = "gbe0"; char* link_name_out = "gbe16"; // Parse args. for (int i = 1; i < argc; i++) { char* arg = argv[i]; if (!strcmp(arg, "--link") !strcmp(arg, "--links")) && i + 1 < argc { // --links <link_name_in> <link_name_out> // link_name_in is ingress port, link_name_out is egress port. // if only one link is provided, it will be used as ingress and // egress port. link_name_in = argv[++i]; link_name_out = link_name_in; } } </pre>

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code (continued)

Single-instance mPIPE (app.c) (continued)	Multi-instance mPIPE (app_gx8072.c) (continued)
<pre> else if (!strcmp(arg, "-n") && i + 1 < argc) { num_packets = atoi(argv[++i]); } else if (!strcmp(arg, "-w") && i + 1 < argc) { num_workers = atoi(argv[++i]); } else if (!strcmp(arg, "--flip")) { flip = true; } else if (!strcmp(arg, "--jumbo")) { jumbo = true; } else { tmc_task_die("Unknown option '%s'.", arg); } } // Determine the available cpus. result = tmc_cpus_get_my_affinity(&cpus); VERIFY(result, "tmc_cpus_get_my_affinity()"); if (tmc_cpus_count(&cpus) < num_workers) tmc_task_die("Insufficient cpus."); // Get the instance. instance = gxio_mpipe_link_instance(link_name); if (instance < 0) tmc_task_die("Link '%s' does not exist.", link_name); // Start the driver. result = gxio_mpipe_init(context, instance); VERIFY(result, "gxio_mpipe_init()"); gxio_mpipe_link_t link; result = gxio_mpipe_link_open(&link, context, link_name, 0); VERIFY(result, "gxio_mpipe_link_open()"); int channel = gxio_mpipe_link_channel(&link); if (jumbo) { // Allow the link to receive jumbo packets. gxio_mpipe_link_set_attr(&link, GXIO_MPIPE_LINK_RECEIVE_JUMBO, 1); } // Allocate some iqueues. iqueues = calloc(num_workers, sizeof(*iqueues)); if (iqueues == NULL) tmc_task_die("Failure in 'calloc()'."); // Allocate some NotifRings. result = gxio_mpipe_alloc_notif_rings(context, num_workers, 0, 0); VERIFY(result, "gxio_mpipe_alloc_notif_rings()"); unsigned int ring = result; </pre>	<pre> if (strstr(argv[i + 1], "gbe")) { link_name_out = argv[++i]; } } else if (!strcmp(arg, "-n") && i + 1 < argc) { num_packets = atoi(argv[++i]); } else if (!strcmp(arg, "-w") && i + 1 < argc) { num_workers = atoi(argv[++i]); } else if (!strcmp(arg, "--flip")) { flip = true; } else if (!strcmp(arg, "--jumbo")) { jumbo = true; } else { tmc_task_die("Unknown option '%s'.", arg); } } // Determine the available cpus. result = tmc_cpus_get_my_affinity(&cpus); VERIFY(result, "tmc_cpus_get_my_affinity()"); if (tmc_cpus_count(&cpus) < num_workers) tmc_task_die("Insufficient cpus."); // Get the instance. instance = gxio_mpipe_link_instance(link_name_in); if (instance < 0) tmc_task_die("Link '%s' does not exist.", link_name_in); // Start the driver. result = gxio_mpipe_init(NULL, instance); VERIFY(result, "gxio_mpipe_init()"); // Get the gxio mPIPE context of the instance just initialized. context[LINK_INGRESS] = GXIO_MPIPE_CONTEXT(instance); gxio_mpipe_link_t link; result = gxio_mpipe_link_open(&link, context[LINK_INGRESS], link_name_in, 0); VERIFY(result, "gxio_mpipe_link_open()"); int channel = gxio_mpipe_link_channel(&link); if (jumbo) { // Allow the link to receive jumbo packets. gxio_mpipe_link_set_attr(&link, GXIO_MPIPE_LINK_RECEIVE_JUMBO, 1); } // Allocate some iqueues. iqueues = calloc(num_workers, sizeof(*iqueues)); if (iqueues == NULL) tmc_task_die("Failure in 'calloc()'."); // Allocate some NotifRings. result = gxio_mpipe_alloc_notif_rings(context[LINK_INGRESS], num_workers, 0, 0); VERIFY(result, "gxio_mpipe_alloc_notif_rings()"); unsigned int ring = result; </pre>

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code (continued)

Single-instance mPIPE (app.c) (continued)	Multi-instance mPIPE (app_gx8072.c) (continued)
<pre> // Init the NotifRings. size_t notif_ring_entries = 512; size_t notif_ring_size = notif_ring_entries * sizeof(gxio_mpipe_idesc_t); size_t needed = notif_ring_size + sizeof(gxio_mpipe_iqueue_t); for (int i = 0; i < num_workers; i++) { tmc_alloc_t alloc = TMC_ALLOC_INIT; tmc_alloc_set_home(&alloc, tmc_cpus_find_nth_cpu(&cpus, i)); // The ring must use physically contiguous memory, but the iqueue // can span pages, so we use "notif_ring_size", not "needed". tmc_alloc_set_pagesize(&alloc, notif_ring_size); void* iqueue_mem = tmc_alloc_map(&alloc, needed); if (iqueue_mem == NULL) tmc_task_die("Failure in 'tmc_alloc_map()'."); gxio_mpipe_iqueue_t* iqueue = iqueue_mem + notif_ring_size; result = gxio_mpipe_iqueue_init(iqueue, context, ring + i, iqueue_mem, notif_ring_size, 0); VERIFY(result, "gxio_mpipe_iqueue_init()"); iqueues[i] = iqueue; } // Allocate a NotifGroup. result = gxio_mpipe_alloc_notif_groups(context, 1, 0, 0); VERIFY(result, "gxio_mpipe_alloc_notif_groups()"); int group = result; // Allocate some buckets. The default mPipe classifier requires // the number of buckets to be a power of two (maximum of 4096). int num_buckets = 1024; result = gxio_mpipe_alloc_buckets(context, num_buckets, 0, 0); VERIFY(result, "gxio_mpipe_alloc_buckets()"); int bucket = result; // Init group and buckets, preserving packet order among flows. gxio_mpipe_bucket_mode_t mode = GXIO_MPIPE_BUCKET_DYNAMIC_FLOW_AFFINITY; result = gxio_mpipe_init_notif_group_and_buckets(context, group, ring, num_workers, bucket, num_buckets, mode); VERIFY(result, "gxio_mpipe_init_notif_group_and_buckets()"); // Initialize the equeue. result = gxio_mpipe_alloc_edma_rings(context, 1, 0, 0); </pre>	<pre> // Init the NotifRings. size_t notif_ring_entries = 512; size_t notif_ring_size = notif_ring_entries * sizeof(gxio_mpipe_idesc_t); size_t needed = notif_ring_size + sizeof(gxio_mpipe_iqueue_t); for (int i = 0; i < num_workers; i++) { tmc_alloc_t alloc = TMC_ALLOC_INIT; tmc_alloc_set_home(&alloc, tmc_cpus_find_nth_cpu(&cpus, i)); // The ring must use physically contiguous memory, but the iqueue // can span pages, so we use "notif_ring_size", not "needed". tmc_alloc_set_pagesize(&alloc, notif_ring_size); void* iqueue_mem = tmc_alloc_map(&alloc, needed); if (iqueue_mem == NULL) tmc_task_die("Failure in 'tmc_alloc_map()'."); gxio_mpipe_iqueue_t* iqueue = iqueue_mem + notif_ring_size; result = gxio_mpipe_iqueue_init(iqueue, context[LINK_INGRESS], ring + i, iqueue_mem, notif_ring_size, 0); VERIFY(result, "gxio_mpipe_iqueue_init()"); iqueues[i] = iqueue; } // Allocate a NotifGroup. result = gxio_mpipe_alloc_notif_groups(context[LINK_INGRESS], 1, 0, 0); VERIFY(result, "gxio_mpipe_alloc_notif_groups()"); int group = result; // Allocate some buckets. The default mPipe classifier requires // the number of buckets to be a power of two (maximum of 4096). int num_buckets = 1024; result = gxio_mpipe_alloc_buckets(context[LINK_INGRESS], num_buckets, 0, 0); VERIFY(result, "gxio_mpipe_alloc_buckets()"); int bucket = result; // Init group and buckets, preserving packet order among flows. gxio_mpipe_bucket_mode_t mode = GXIO_MPIPE_BUCKET_DYNAMIC_FLOW_AFFINITY; result = gxio_mpipe_init_notif_group_and_buckets(context[LINK_INGRESS], group, ring, num_workers, bucket, num_buckets, mode); VERIFY(result, "gxio_mpipe_init_notif_group_and_buckets()"); // Initialize the equeue. instance = gxio_mpipe_link_instance(link_name_out); result = gxio_mpipe_init(NULL, instance); VERIFY(result, "gxio_mpipe_init()"); context[LINK_EGRESS] = GXIO_MPIPE_CONTEXT(instance); if (strcmp(link_name_in, link_name_out)) { result = gxio_mpipe_link_open(&link, context[LINK_EGRESS], link_name_out, 0); VERIFY(result, "gxio_mpipe_link_open()"); channel = gxio_mpipe_link_channel(&link); if (jumbo) { // Allow the link to receive jumbo packets. gxio_mpipe_link_set_attr(&link, GXIO_MPIPE_LINK_RECEIVE_JUMBO, 1); } } </pre>

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code (continued)

Single-instance mPIPE (app.c) (continued)	Multi-instance mPIPE (app_gx8072.c) (continued)
<pre> VERIFY(result, "gxio_mpipe_alloc_edma_rings"); uint ering = result; size_t edescs_size = equeue_entries * sizeof(gxio_mpipe_edesc_t); tmc_alloc_t edescs_alloc = TMC_ALLOC_INIT; tmc_alloc_set_pagesize(&edescs_alloc, edescs_size); void* edescs = tmc_alloc_map(&edescs_alloc, edescs_size); if (edescs == NULL) tmc_task_die("Failed to allocate equeue memory."); result = gxio_mpipe_equeue_init(equeue, context, ering, channel, edescs, edescs_size, 0); VERIFY(result, "gxio_equeue_init()"); // Use enough small/large buffers to avoid ever getting "idesc->be". unsigned int num_bufs = equeue_entries + num_workers * notif_ring_entries; // Allocate small/large/jumbo buffer stacks. result = gxio_mpipe_alloc_buffer_stacks(context, jumbo ? 3 : 2, 0, 0); VERIFY(result, "gxio_mpipe_alloc_buffer_stacks()"); int stack_idx = result; // Initialize small/large stacks. create_stack(context, stack_idx + 0, GXIO_MPIPE_BUFFER_SIZE_128, num_bufs); create_stack(context, stack_idx + 1, GXIO_MPIPE_BUFFER_SIZE_1664, num_bufs); if (jumbo) { // Initialize jumbo stack. We use 16K buffers, because 4K buffers // are too small, and 10K buffers can induce "false chaining". We // use only 4 buffers per worker, because they use a lot of memory, // and the risk of "idesc->be" is low. create_stack(context, stack_idx + 2, GXIO_MPIPE_BUFFER_SIZE_16384, num_workers * 4); // Make sure all "possible" jumbo packets can be egressed safely. result = gxio_mpipe_equeue_set_snf_size(equeue, 10384); VERIFY(result, "gxio_mpipe_equeue_set_snf_size()"); } // Register for packets. gxio_mpipe_rules_t rules; gxio_mpipe_rules_init(&rules, context); gxio_mpipe_rules_begin(&rules, bucket, num_buckets, NULL); result = gxio_mpipe_rules_commit(&rules); VERIFY(result, "gxio_mpipe_rules_commit()"); tmc_sync_barrier_init(&sync_barrier, num_workers); tmc_spin_barrier_init(&spin_barrier, num_workers); pthread_t threads[num_workers]; for (int i = 1; i < num_workers; i++) { if (pthread_create(&threads[i], NULL, main_aux, (void*)(intptr_t)i) != 0) tmc_task_die("Failure in 'pthread_create()'"); } </pre>	<pre> result = gxio_mpipe_alloc_edma_rings(context[LINK_EGRESS], 1, 0, 0); VERIFY(result, "gxio_mpipe_alloc_edma_rings"); uint ering = result; size_t edescs_size = equeue_entries * sizeof(gxio_mpipe_edesc_t); tmc_alloc_t edescs_alloc = TMC_ALLOC_INIT; tmc_alloc_set_pagesize(&edescs_alloc, edescs_size); void* edescs = tmc_alloc_map(&edescs_alloc, edescs_size); if (edescs == NULL) tmc_task_die("Failed to allocate equeue memory."); result = gxio_mpipe_equeue_init(equeue, context[LINK_EGRESS], ering, channel, edescs, edescs_size, 0); VERIFY(result, "gxio_equeue_init()"); // Use enough small/large buffers to avoid ever getting "idesc->be". unsigned int num_bufs = equeue_entries + num_workers * notif_ring_entries; // Allocate small/large/jumbo buffer stacks. result = gxio_mpipe_alloc_buffer_stacks(context[LINK_INGRESS], jumbo ? 3 : 2, 0, 0); VERIFY(result, "gxio_mpipe_alloc_buffer_stacks()"); int stack_idx = result; // Initialize small/large stacks. create_stack(context[LINK_INGRESS], stack_idx + 0, GXIO_MPIPE_BUFFER_SIZE_128, num_bufs); create_stack(context[LINK_INGRESS], stack_idx + 1, GXIO_MPIPE_BUFFER_SIZE_1664, num_bufs); if (jumbo) { // Initialize jumbo stack. We use 16K buffers, because 4K buffers // are too small, and 10K buffers can induce "false chaining". We // use only 4 buffers per worker, because they use a lot of memory, // and the risk of "idesc->be" is low. create_stack(context[LINK_INGRESS], stack_idx + 2, GXIO_MPIPE_BUFFER_SIZE_16384, num_workers * 4); // Make sure all "possible" jumbo packets can be egressed safely. result = gxio_mpipe_equeue_set_snf_size(equeue, 10384); VERIFY(result, "gxio_mpipe_equeue_set_snf_size()"); } // Register for packets. gxio_mpipe_rules_t rules; gxio_mpipe_rules_init(&rules, context[LINK_INGRESS]); gxio_mpipe_rules_begin(&rules, bucket, num_buckets, NULL); result = gxio_mpipe_rules_commit(&rules); VERIFY(result, "gxio_mpipe_rules_commit()"); tmc_sync_barrier_init(&sync_barrier, num_workers); tmc_spin_barrier_init(&spin_barrier, num_workers); pthread_t threads[num_workers]; for (int i = 1; i < num_workers; i++) { if (pthread_create(&threads[i], NULL, main_aux, (void*)(intptr_t)i) != 0) tmc_task_die("Failure in 'pthread_create()'"); } </pre>

Table 5-3. Comparing Single and Multiple Instance mPIPE Forwarding Code (continued)

Single-instance mPIPE (app.c) (continued)	Multi-instance mPIPE (app_gx8072.c) (continued)
<pre> (void)main_aux((void*)(intptr_t)0); for (int i = 1; i < num_workers; i++) { if (pthread_join(threads[i], NULL) != 0) tmc_task_die("Failure in 'pthread_join()'"); } // FIXME: Wait until pending egress is "done". for (int i = 0; i < 1000000; i++) __insn_mfspr(SCR_PASS); return 0; } </pre>	<pre> (void)main_aux((void*)(intptr_t)0); for (int i = 1; i < num_workers; i++) { if (pthread_join(threads[i], NULL) != 0) tmc_task_die("Failure in 'pthread_join()'"); } // FIXME: Wait until pending egress is "done". for (int i = 0; i < 1000000; i++) __insn_mfspr(SCR_PASS); return 0; } </pre>

CHAPTER 6 MDE SOFTWARE WITH NO MIGRATION ISSUES

This chapter lists software that works the same on the *TILEPro*[™] and the *TILE-Gx*[™] processor family and has no migration issues:

- [Section 6.1 “Tools That are Identical”](#)
- [Section 6.2 “Tools That are the Same From the User Perspective”](#)

6.1 Tools That are Identical

These tools are the same on the *TILEPro* and the *TILE-Gx* processor family:

- `sbim, tile-sbim` (Tile Processor SROM Boot Image Manager)
- `tile-console` (Tile Processor Console)
- `tile-gprof` (Tile Processor Profiler)
- `tile-mkboot` (Tile Processor Boot Image Creator)
- `tile-trace-view` (Tile Processor Trace Viewer)

6.2 Tools That are the Same From the User Perspective

These tools are the same from the user perspective, though they are implemented somewhat differently on the two processor families:

- `mcstat` (Tile Processor Memory Controller Statistics). Memory bandwidth profiler.
- `tile-ld` (Tile Processor Linker). The version for each processor family will only accept and produce executables for that processor family.
- `tile-mkrom` (Tile Processor Bootstream Formatter). This tool will check for executables for the correct processor family.
- `tile-monitor` (Tile Processor Monitor). This tool checks for executables for the correct processor family.

INDEX

Numerics

3GPP (3rd Generation Partnership Project) [4](#)

A

about this guide [v](#)

Advanced Encryption Standard (AES) [4](#)

API libraries [1](#)

C

compression operations [1](#)

context object [3](#)

conventions

notation [vi](#)

crypto acceleration

protocols supported [4](#)

CSUM_BYTES [15](#)

CSUM_SEED [15](#)

CSUM_START [15](#)

customer support [vi](#)

D

DDR3 timing [5](#)

DES (Data Encryption Standard) [4](#)

Diffie-Hellman key exchange [4](#)

DMA [5, 6](#)

double-XAUI [1](#)

DSA [4](#)

DTLS [1](#)

E

ECC [4](#)

G

GPIO device API

gxio/gpio.h [3](#)

GP_SQN [15](#)

GXIO library [3](#)

gxio/gpio.h [3](#)

gxio/mica.h [3](#)

gxio/mpipe.h [3](#)

gxio/trio.h [3](#)

gxio_mpipe_equeue_put() [14](#)

gxio_mpipe_equeue_put_at() [14](#)

gxio_mpipe_equeue_reserve() [14](#)

gxio_mpipe_iqueue_get() [13](#)

gxio_mpipe_iqueue_peek() [13](#)

gxio_mpipe_iqueue_try_peek() [13, 14](#)

H

high speed crypto operations [1](#)

HMAC [4](#)

I

I/O shims [3](#)

IDE online help [vi](#)

iDMA [15](#)

info pages [vi](#)

installation process

new procedures [5](#)

intended audience [v](#)

IPI mechanism [1](#)

IPsec [1, 4](#)

K

KASUMI [4](#)

M

MACsec [1, 4](#)

man pages [vi](#)

mcstat [27](#)

MD5 [4](#)

MDE Document Index [v](#)

location of [vi](#)

MDE documentation

location of [v](#)

memory mapped I/O interface

See MMIO

MiCA [1, 4](#)

MiCA device API

gxio/mica.h [3](#)

miscellaneous software changes [6](#)

MMIO [1, 6](#)

control registers [3](#)

pages [1](#)

mPIPE [1, 4](#)

mPIPE device API

gxio/mpipe.h [3](#)

Multicore iMesh Coprocessing Accelerator

See MiCA

multicore Programmable Intelligent Packet Engine

See mPIPE

N

netio_get_packet() [13, 14](#)
 netio_send_packet() [14](#)
 notation conventions [vi](#)
 NotifRingIDX [15](#)

P

PacketSQN [16](#)
 PCIe
 drivers [6](#)
 interfaces [1](#)
 PIO [6](#)
 programming environments
 supported [1](#)
 public key [1](#)

R

RC4 [4](#)
 recompilation
 need for [13](#)
 RNG [4](#)
 root complex driver [6](#)
 RPMs [5](#)
 RSA [4](#)

S

sbim [27](#)
 SHA-1 [4](#)
 SHA-2 [4](#)
 simulator [5](#)
 software features
 for 2.0 release [3](#)
 SRTP [1](#)
 SSL [1, 4](#)
 StreamIO [5](#)
 StreamIO data [1](#)
 support

technical or customer [vi](#)

T

technical support [vi](#)
 Tile Multicore Components
 See TMC
 tile-console [27](#)
 tile-gcc/g++
 changes in [5](#)
 tile-gprof [27](#)
 TILE-Gx
 PCIe drivers [6](#)
 processor family features [1](#)
 tile-ld [27](#)
 tile-mkboot [27](#)
 tile-mkrom [27](#)
 tile-monitor [27](#)
 TILEPro
 processor family features [1](#)
 tile-sbim [27](#)
 Tile-to-Tile DMA
 unsupported [6](#)
 tile-trace-view [27](#)
 TLS [1](#)
 TMC [1](#)
 TRIO [1, 5](#)
 TRIO device API
 gxio/trio.h [3](#)
 Triple DES (3DES) [4](#)

W

wireless security [1](#)

Z

Zero overhead Linux
 See ZOL
 ZOL [2](#)